

```

const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*' } });

app.use(express.static('public'));

// — Game Constants —————
const WORLD_W = 2400;
const WORLD_H = 1800;
const SHIP_SPEED = 3.2;
const BULLET_SPEED = 9;
const SHIP_RADIUS = 36;
const SHRINK_INTERVAL = 30000; // 30s between shrinks
const SHRINK_AMOUNT = 180;
const SPAWN_PADDING = 200;
const RELOAD_TIME = 1500; // ms

// — State —————
let players = {};
let bullets = [];
let loot = [];
let gameState = 'lobby'; // lobby | countdown | playing | ended
let safeZone = { x: WORLD_W / 2, y: WORLD_H / 2, r: Math.min(WORLD_W, WORLD_H) / 2 };
let nextShrink = Date.now() + SHRINK_INTERVAL;
let winner = null;
let bulletId = 0;

const SHIP_COLORS = [
  '#e74c3c', '#e67e22', '#f1c40f', '#2ecc71', '#1abc9c',
  '#3498db', '#9b59b6', '#e91e8c', '#00bcd4', '#ff5722',
  '#8bc34a', '#ff9800', '#673ab7', '#f06292', '#26c6da',
  '#66bb6a', '#ffa726', '#ab47bc', '#ef5350', '#42a5f5',
];
let colorIndex = 0;

const PIRATE_NAMES = [
  'Blackbeard', 'Calico Jack', 'Anne Bonny', 'Mary Read', 'Bartholomew',
  'Long John', 'Dead Eye', 'Sea Wolf', 'Iron Claw', 'Skull Pete',
  'Rum Runner', 'The Kraken', 'Saltbeard', 'Jolly Rogue', 'Whirlpool',
  'Tempest', 'The Eel', 'Barnacle', 'Corsair', 'Buccaneer',

```

```

];

// ——— Helpers —————
function randomSpawn() {
  const angle = Math.random() * Math.PI * 2;
  const r = Math.random() * (safeZone.r - SPAWN_PADDING);
  return {
    x: safeZone.x + Math.cos(angle) * r,
    y: safeZone.y + Math.sin(angle) * r,
  };
}

function spawnLoot() {
  loot = [];
  for (let i = 0; i < 18; i++) {
    const pos = randomSpawn();
    loot.push({
      id: i,
      x: pos.x,
      y: pos.y,
      type: Math.random() < 0.3 ? 'health' : 'gold',
      value: Math.random() < 0.3 ? 1 : Math.floor(Math.random() * 3 + 1) * 10,
    });
  }
}

function getLeaderboard() {
  return Object.values(players)
    .sort((a, b) => (b.kills * 100 + b.score) - (a.kills * 100 + a.score))
    .map(p => ({ name: p.name, color: p.color, kills: p.kills, score: p.score, al
}

function getPublicState() {
  return {
    gameState,
    safeZone,
    players: Object.fromEntries(
      Object.entries(players).map(([id, p]) => [id, {
        x: p.x, y: p.y, angle: p.angle, hp: p.hp, alive: p.alive,
        name: p.name, color: p.color, score: p.score, kills: p.kills,
        reloadReady: p.reloadReady,
      }])
    ),
    bullets: bullets.map(b => ({ id: b.id, x: b.x, y: b.y, ownerId: b.ownerId })),
    loot,
    winner,
    playerCount: Object.keys(players).length,
  }
}

```

```

    };
}

// — Game Flow —————
function startCountdown() {
    gameState = 'countdown';
    io.emit('gameState', getPublicState());

    let count = 5;
    const interval = setInterval(() => {
        count--;
        io.emit('countdown', count);
        if (count <= 0) {
            clearInterval(interval);
            startGame();
        }
    }, 1000);
}

function startGame() {
    gameState = 'playing';
    winner = null;
    safeZone = { x: WORLD_W / 2, y: WORLD_H / 2, r: Math.min(WORLD_W, WORLD_H) / 2 };
    nextShrink = Date.now() + SHRINK_INTERVAL;
    bullets = [];
    spawnLoot();

    Object.values(players).forEach(p => {
        const pos = randomSpawn();
        p.x = pos.x;
        p.y = pos.y;
        p.hp = 3;
        p.score = 0;
        p.kills = 0;
        p.alive = true;
        p.angle = Math.random() * Math.PI * 2;
        p.reloadReady = true;
        p.stormTimer = 0;
    });

    io.emit('gameStart', getPublicState());
}

function endGame(winnerPlayer) {
    gameState = 'ended';
    winner = winnerPlayer
    ? { name: winnerPlayer.name, color: winnerPlayer.color, score: winnerPlayer.s

```

```

        : null;
    io.emit('gameEnd', { winner, players: getLeaderboard() });
}

function checkWin() {
    const alive = Object.values(players).filter(p => p.alive);
    if (alive.length <= 1) {
        endGame(alive[0] || null);
        return true;
    }
    return false;
}

// — Game Loop —————
setInterval(() => {
    if (gameState !== 'playing') return;

    const now = Date.now();

    // Shrink safe zone
    if (now > nextShrink && safeZone.r > 200) {
        safeZone.r = Math.max(200, safeZone.r - SHRINK_AMOUNT);
        nextShrink = now + SHRINK_INTERVAL;
        io.emit('zoneShrink', safeZone);
    }

    // Move players
    Object.values(players).forEach(p => {
        if (!p.alive || !p.input) return;
        const { dx, dy } = p.input;
        if (dx !== 0 || dy !== 0) {
            const len = Math.sqrt(dx * dx + dy * dy);
            p.x += (dx / len) * SHIP_SPEED;
            p.y += (dy / len) * SHIP_SPEED;
            p.angle = Math.atan2(dy, dx) + Math.PI / 2;
        }
        p.x = Math.max(0, Math.min(WORLD_W, p.x));
        p.y = Math.max(0, Math.min(WORLD_H, p.y));
    });

    // Move bullets + hit detection
    bullets = bullets.filter(b => {
        b.x += Math.cos(b.angle) * BULLET_SPEED;
        b.y += Math.sin(b.angle) * BULLET_SPEED;
        b.life--;

        if (b.life <= 0) return false;
    });

```

```

if (b.x < 0 || b.x > WORLD_W || b.y < 0 || b.y > WORLD_H) return false;

for (const [id, p] of Object.entries(players)) {
  if (!p.alive || id === b.ownerId) continue;
  const dx = p.x - b.x, dy = p.y - b.y;
  if (Math.sqrt(dx * dx + dy * dy) < SHIP_RADIUS) {
    p.hp--;
    const shooter = players[b.ownerId];
    if (shooter) shooter.score += 25;
    io.emit('hit', { targetId: id, shooterId: b.ownerId, hp: p.hp });

    if (p.hp <= 0) {
      p.alive = false;
      p.hp = 0;
      if (shooter) { shooter.kills++; shooter.score += 100; }
      io.emit('playerDied', { id, killedBy: shooter?.name || 'The Sea' });
      checkWin();
    }
    return false;
  }
}
return true;
});

// Storm damage
Object.entries(players).forEach(([id, p]) => {
  if (!p.alive) return;
  const dx = p.x - safeZone.x, dy = p.y - safeZone.y;
  if (Math.sqrt(dx * dx + dy * dy) > safeZone.r) {
    p.stormTimer = (p.stormTimer || 0) + 1;
    if (p.stormTimer % 120 === 0) {
      p.hp = Math.max(0, p.hp - 1);
      io.emit('hit', { targetId: id, shooterId: 'storm', hp: p.hp });
      if (p.hp <= 0) {
        p.alive = false;
        io.emit('playerDied', { id, killedBy: 'The Storm' });
        checkWin();
      }
    }
  }
  else {
    p.stormTimer = 0;
  }
});

// Loot pickup
loot = loot.filter(l => {
  for (const [id, p] of Object.entries(players)) {

```

```

    if (!p.alive) continue;
    const dx = p.x - l.x, dy = p.y - l.y;
    if (Math.sqrt(dx * dx + dy * dy) < 40) {
      if (l.type === 'health') p.hp = Math.min(3, p.hp + 1);
      else p.score += l.value;
      io.emit('lootPickup', { playerId: id, lootId: l.id, type: l.type, value:
        return false;
      }
    }
    return true;
  });

  io.emit('tick', getPublicState());
}, 1000 / 60);

// — Socket Events —————
io.on('connection', (socket) => {
  console.log('Connected:', socket.id);

  socket.on('join', ({ name }) => {
    const cleanName = (name || '').trim().slice(0, 16) || PIRATE_NAMES[Math.floor
    const pos = randomSpawn();

    players[socket.id] = {
      id: socket.id,
      name: cleanName,
      color: SHIP_COLORS[colorIndex++ % SHIP_COLORS.length],
      x: pos.x, y: pos.y,
      angle: 0, aimAngle: 0,
      hp: 3, alive: true,
      score: 0, kills: 0,
      input: { dx: 0, dy: 0 },
      reloadReady: true,
      stormTimer: 0,
    };

    socket.emit('joined', { id: socket.id, player: players[socket.id], worldW: WO
    socket.emit('gameState', getPublicState());
    io.emit('playerJoined', { name: cleanName, count: Object.keys(players).length
    console.log(`${cleanName} joined. Total: ${Object.keys(players).length}`);
  });

  socket.on('input', (input) => {
    if (players[socket.id]) players[socket.id].input = input;
  });

  socket.on('fire', ({ angle }) => {

```

```

    const p = players[socket.id];
    if (!p || !p.alive || !p.reloadReady || gameState !== 'playing') return;
    p.reloadReady = false;
    bullets.push({ id: bulletId++, x: p.x, y: p.y, angle, ownerId: socket.id, lif
    io.emit('fired', { ownerId: socket.id, x: p.x, y: p.y });
    setTimeout(() => { if (players[socket.id]) players[socket.id].reloadReady = t
  });

  socket.on('startGame', () => {
    if (gameState === 'lobby' || gameState === 'ended') {
      if (Object.keys(players).length >= 1) startCountdown();
    }
  });

  socket.on('disconnect', () => {
    const p = players[socket.id];
    if (p) {
      console.log(`${p.name} disconnected.`);
      io.emit('playerLeft', { name: p.name, count: Object.keys(players).length -
      delete players[socket.id];
    }
    if (gameState === 'playing') checkWin();
  });
});

// ——— Start —————
const PORT = process.env.PORT || 3000;

server.listen(PORT, () => {
  // Works locally and on Railway/Render/any cloud host
  const publicUrl = process.env.RAILWAY_PUBLIC_DOMAIN
    ? `https://${process.env.RAILWAY_PUBLIC_DOMAIN}`
    : `http://localhost:${PORT}`;

  console.log(`\n🏴‍☠️ DEAD RECKONING — Pirate Battle Royale`);
  console.log('=====');
  console.log(`\n Host display:   ${publicUrl}/host`);
  console.log(`\n Players join:   ${publicUrl}`);
  console.log(`\n=====
});

```